



## SISTEMAS OPERATIVOS - SOLEMNE 01 PRÁCTICO PARTE 2 FORMA B

A continuación, se presenta la parte final de la prueba solemne 1, parte práctica, esta se dividirá, en dos entregables, un intermedio, que se realiza y sube, durante la hora de clases y el final, que además contempla insumos de la primera parte, cabe señalar que la nota de la prueba solemne se compondrá de la siguiente forma:

- 50% parte teórica (rendida el miércoles 02-09)
- 50% parte práctica, la que se compone de la siguiente forma:
  - 50% nota parte 01
  - 10% nota parte 02 (entrega intermedia)
  - 40% nota parte 02 (entrega final)

Recuerden que todo estudiante del equipo que no este presente durante las partes 01 y 02 entrega intermedia, serán evaluados con nota 1.0, por otra parte si algún miembro del equipo no colocadora en cualquiera de las entregas, es posible enviar un correo al docente indicando esta situación, solicitando, excluir al estudiantes de las entregas, quedando a criterio del docente, en base a las evidencias entregadas en el correo, si excluye o no al estudiante, el plazo máximo para el envío de este correo es el 11-09-2026 hasta las 20:00 horas.

### Contexto

El centro de monitoreo ambiental debe trasladar su solución a un ambiente Debian. En este entorno debe ejecutar el procesador secuencial y concurrente de mediciones, comprobar la consistencia de los resultados y desarrollar un módulo para clasificar informes y separar alertas por indicador, el sistema final debe organizar informes sin alertas, con alertas y críticos; conservar el resumen general; separar alertas de temperatura, humedad, PM2.5 y ruido; generar un inventario JSON y mantener una bitácora de gestión.

### Entregable intermedio: hito presencial

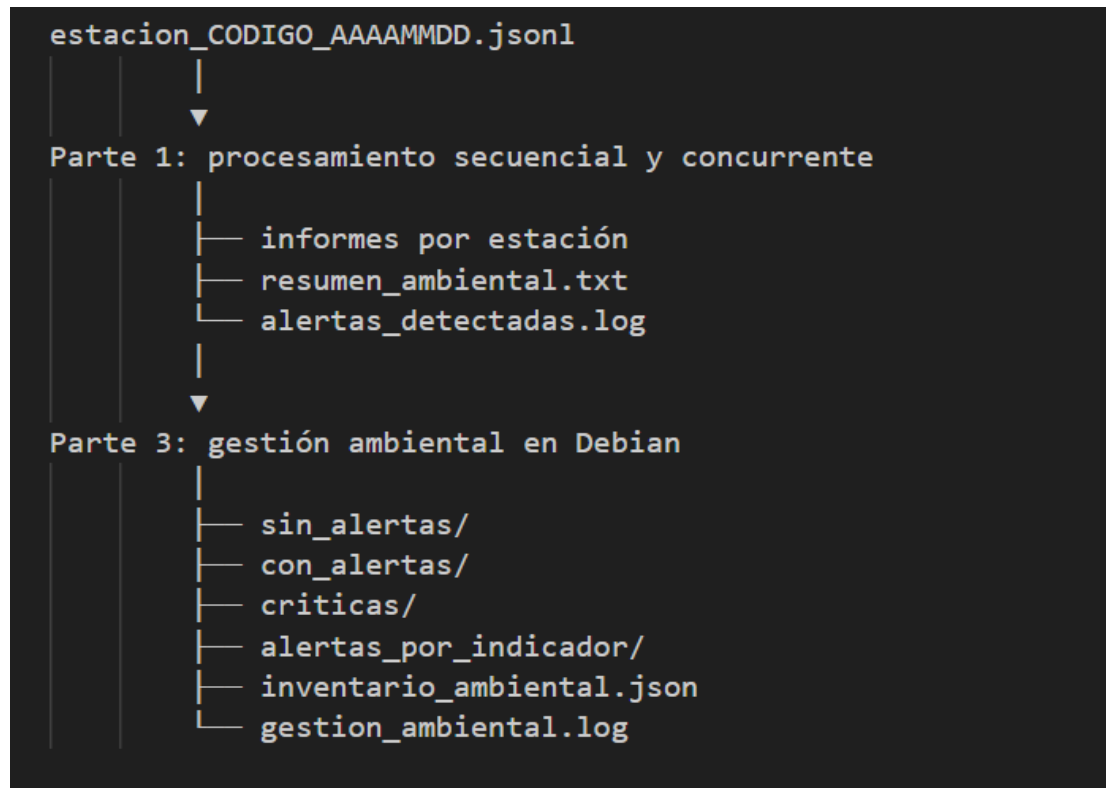
El tiempo máximo, para esta entrega es de 30 minutos, este

Puntaje: Según rúbrica al final de este documento

Archivo requerido: hito\_parte2hp\_equipoXX.pdf

El documento debe tener entre una y dos páginas e incluir:

- Integrantes, sección y lenguaje usado en Parte 1.
- Objetivo de secuencial, concurrente y gestor\_incidentes.
- Flujo planificado:



- Configuración planificada de la máquina virtual:
  - Hipervisor.
  - RAM.
  - Procesadores virtuales.
  - Disco virtual.
  - Red.
- Paquetes requeridos según el lenguaje.
- Tres comandos de GNU/Linux para observar procesos, memoria o archivos.
- Riesgo identificado y medida de mitigación:
  - Ejemplo: sobrescritura de un reporte; se evitará validando la existencia del destino y aplicando sufijo correlativo.
- Cinco evidencias que serán incluidas en el informe final.

Entregable final: hito asincrónico

## 1. Instalar Debian

Instale Debian 13 “trixie” estable, de 64 bits o versión similar siempre que sea Debian, puede hacerlo en VirtualBox, VMware u otro hipervisor, la versión estable vigente se distribuye mediante la imagen debian-13.6.0-amd64-netinst.iso, la máquina virtual debe tener:

| Recurso                | Mínimo             | Recomendado          |
|------------------------|--------------------|----------------------|
| RAM                    | 2 GB               | 4 GB                 |
| Procesadores virtuales | 2                  | 2 o 4                |
| Disco virtual          | 20 GB              | 25 GB                |
| Red                    | NAT                | NAT con conectividad |
| Sistema                | Debian 13, 64 bits | Debian 13, 64 bits   |

Documento descarga de ISO, creación de VM, configuración de recursos, instalación, usuario, particionado sobre disco virtual, primer inicio y conectividad.

## 2. Preparar ambiente

a) Ejecute:

```
sudo apt update  
sudo apt upgrade -y
```

b) Instale el entorno del lenguaje usado previamente.

```
Python  
sudo apt install -y python3 python3-venv python3-pip procps psmisc tree  
python3 --version  
  
Java  
sudo apt install -y default-jdk procps psmisc tree  
java --version  
javac --version  
  
C  
sudo apt install -y build-essential procps psmisc tree  
gcc --version  
make --version
```

c) Ejecutar Parte 1 en Debia, ejecute la versión secuencial y concurrente con el mismo conjunto de archivos.jsonl, deben coincidir:

- Archivos procesados.
- Mediciones válidas.
- Mediciones inválidas.
- Alertas detectadas.
- PM2.5 máximo global.
- Ruido máximo global.

d) Desarrollar gestor de incidencias, en Debian:

src/gestor\_incidencias.[py|java|c]

La estructura debe quedar:

```
solemne_so_equipoXX/  
├─ entrada/  
├─ salida/  
│   ├─ informe_STG01_20260902.txt  
│   └─ resumen_ambiental.txt  
├─ alertas/  
│   └─ alertas_detectadas.log  
├─ gestion_ambiental/  
│   ├─ sin_alertas/  
│   ├─ con_alertas/  
│   ├─ criticas/  
│   ├─ alertas_por_indicador/  
│   │   ├─ temperatura/  
│   │   ├─ humedad/  
│   │   ├─ pm25/  
│   │   └─ ruido/  
│   └─ inventario_ambiental.json  
├─ logs/  
│   └─ gestion_ambiental.log  
├─ src/  
│   ├─ secuencial.[py|java|c]  
│   ├─ concurrente.[py|java|c]  
│   └─ gestor_incidencias.[py|java|c]  
├─ evidencias/  
└─ README.md
```

#### Entregable final Versión

- Informe PDF de 8 a 12 páginas.
- Código fuente de Parte 1 y del gestor de cierre.



- Archivos de entrada, reportes, consolidado, inventario y bitácora.
- Capturas y salidas de comandos.
- README final con instrucciones reproducibles.

El gestor debe:

1. Detectar informes informe\_CODIGO\_AAAAMMDD.txt.
2. Leer la cantidad de alertas de cada informe.
3. Mover informes con 0 alertas a sin\_alertas/.
4. Mover informes con 1 a 3 alertas a con\_alertas/.
5. Mover informes con 4 o más alertas a criticas/.
6. Evitar sobrescritura mediante nombres \_v1, \_v2, etc.
7. Copiar salida/resumen\_ambiental.txt como:  
gestion\_ambiental/resumen\_resguardado.txt.
8. Leer alertas/alertas\_detectadas.log.
9. Separar alertas por indicador en archivos de temperatura, humedad, PM2.5 y ruido.
10. Registrar alertas inválidas o desconocidas en la bitácora.
11. Generar gestion\_ambiental/inventario\_ambiental.json.
12. Generar logs/gestion\_ambiental.log.
13. Controlar al menos un error sin detener el procesamiento completo.

e) Observar procesos y sistema de archivos

Durante la ejecución de la versión concurrente del Monitor de Estaciones Ambientales, obtenga evidencia de los procesos, recursos y estructura de archivos generados en Debian. Ejecute comandos como los siguientes:

```
ps -eo pid,ppid,stat,%cpu,%mem,rss,vsz,cmd | grep -E 'python3|java|concurrente'
pstree -p
free -h
df -h
du -sh ~/solemne_so_equipoXX
find ~/solemne_so_equipoXX -type f | sort
ls -lah salida alertas gestion_ambiental logs
stat logs/gestion_ambiental.log
```

Explique:

- PID del proceso principal: identifique el identificador del proceso que ejecuta la versión concurrente.
- PPID del proceso principal: identifique el proceso padre; normalmente corresponde a la terminal, consola o entorno que inició la ejecución.
- Estado del proceso: indique el estado observado en la columna STAT y explique su significado según la salida obtenida.



- Uso de CPU: registre e interprete el porcentaje de CPU utilizado durante la ejecución concurrente.
- Uso de memoria: registre e interprete el porcentaje de memoria indicado por el sistema.
- RSS: identifique la memoria residente utilizada por el proceso, es decir, la porción de memoria que se encuentra cargada físicamente en RAM.
- VSZ: identifique el tamaño de memoria virtual asociado al proceso, el cual puede incluir espacios reservados, bibliotecas y regiones que no están necesariamente residentes en memoria física.
- Memoria del sistema: utilice `free -h` para describir la memoria disponible, utilizada y la memoria destinada a caché, según la salida observada.
- Espacio disponible: utilice `df -h` para identificar el sistema de archivos donde se encuentra el proyecto y el espacio libre disponible.
- Tamaño del proyecto: utilice `du -sh ~/solemne_so_equipoXX` para registrar el espacio ocupado por todos los archivos del proyecto.
- Estructura generada: utilice `find` y `ls -lah` para demostrar:
  - Informes por estación ubicados en `gestion_ambiental/sin_alertas/`, `gestion_ambiental/con_alertas/` o `gestion_ambiental/criticas/`.
  - Archivos de alertas ubicados en las carpetas de `alertas_por_indicador/`.
  - Existencia del inventario `gestion_ambiental/inventario_ambiental.json`.
  - Existencia del respaldo `gestion_ambiental/resumen_resguardado.txt`.
  - Existencia de logs `gestion_ambiental.log`.
- Metadatos de la bitácora, utilice `"stat logs/gestion_ambiental.log"`, para registrar y explicar:
  - Tipo de archivo.
  - Tamaño.
  - Permisos.
  - Usuario propietario.
  - Grupo propietario.
  - Fechas de acceso, modificación y cambio de estado, cuando estén disponibles.
- Diferencia entre mover y copiar: explique por qué los informes deben moverse a sus carpetas de clasificación, mientras que `resumen_ambiental.txt` debe conservarse en `salida/` y, además, copiarse como `resumen_resguardado.txt`.

### Entregable final Versión

- Informe PDF de 8 a 12 páginas.
- Código fuente de Parte 1 y del gestor de incidencias.
- Archivos de entrada, informes, resumen, alertas, inventario, log y evidencias.
- Capturas de instalación y ejecución en Debian.
- README reproducible.

## Rubrica

| Criterio                              | Destacado (71% a 100%)                                                                         | Adecuado (41% a 70%)                                       | Básico (21% a 40%)                            | Insuficiente (0% a 20%)          | Puntaje   |
|---------------------------------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------|-----------------------------------------------|----------------------------------|-----------|
| Hito presencial                       | Planificación completa, viable y contextualizada en monitoreo ambiental                        | Hito correcto con omisiones menores                        | Hito genérico o parcialmente relacionado      | Sin hito o sin comprensión       | 5         |
| Instalación Debian                    | Debian funcional, instalación y configuración de VM documentadas claramente                    | Debian operativo con evidencia suficiente                  | Instalación parcial o escasamente documentada | Sin Debian funcional             | 8         |
| Ejecución Parte 1                     | Ejecuta ambas versiones y verifica métricas ambientales consistentes                           | Ejecuta ambas con evidencia parcial                        | Ejecuta una sola o comparación incompleta     | No ejecuta Parte 1               | 8         |
| Gestor de incidencias y archivos      | Clasifica informes, separa alertas, respalda resumen, inventaría y registra todo correctamente | Cumple la mayor parte de las funciones con errores menores | Gestión parcial, frágil o inconsistente       | No implementa gestor funcional   | 15        |
| Procesos, sistema de archivos y error | Evidencia e interpretación de procesos, recursos, metadatos, permisos y error controlado       | Evidencia suficiente con análisis parcial                  | Evidencia limitada o error mal manejado       | Sin evidencia ni manejo de error | 8         |
| Informe y README                      | Informe claro, técnico, trazable y reproducible                                                | Documento suficiente con pequeñas omisiones                | Informe superficial o README incompleto       | Entregables insuficientes        | 6         |
| <b>Total, Parte 3</b>                 |                                                                                                |                                                            |                                               |                                  | <b>50</b> |